COMP5523 · TEXT-TO-IMAGE RETRIEVAL · MEME-CAP

Meme retrieval: CLIP vs CLIP+NLP vs VLM

A detailed walkthrough of the system architecture, the build and query pipelines of three retrieval strategies, and the evaluation that separates them.

A · clip — pixels only

B · clip_nlp — CLIP + human annotations

C · vlm — CLIP + Qwen2.5-VL descriptions

GALLERY

6,378 images

from 6,382 annotated memes

BENCHMARK

240 queries

80 visual · 80 intent · 80 metaphor

MODELS

ViT-L-14 · BGE-small

+ Qwen2.5-VL-3B, offline

HEADLINE MAP

0.4395 · 0.7608 · 0.4519

A · B · C on the frozen set

THE DIAGRAM SET, ONE DIAGRAM PER PAGE

Diagram 1
System architecture

Diagram 2
The three strategies side by side

Diagram 3
Offline build flow

Diagram 4
Online query flow

Diagram 5
Evaluation pipeline

Generated from the repository — every count, weight and metric on the following pages is read from the code and the report files at build time.

Contents

1	Overview	3
2	The dataset and the gallery	4
3	System architecture	6
4	Strategy A — CLIP only	10
5	Strategy B — CLIP + annotation NLP	11
6	Strategy C — VLM descriptions	13
7	The three strategies side by side	15
8	Offline build flow	18
9	Online query flow	21
10	Evaluation	24
11	Results	27
12	Operating and configuration	30
13	Reproducing the results	32
A	File map	34
B	Where every figure comes from	35
C	Glossary	36

1 Overview

One gallery, three representations of the same 6,378 images, one frozen benchmark, one API. The only thing that changes between strategies is which text — human annotation, model-generated description, or none at all — stands for an image at query time.

Meme retrieval is a hard case for text-to-image search. A meme is usually a photograph or a drawing with overlaid text, and the interesting part is rarely the scene: it is the joke, the implied meaning, the metaphor. A model that aligns pixels with captions can describe *a man in a suit shaking hands with another man* and still miss that the image means *one company acquiring another and pretending it was a merger*. The Meme-Cap dataset exists to make that gap measurable, because each meme carries human-written literal captions and intended-meaning captions, and 5,659 of the 6,378 also carry metaphor/meaning pairs.

The system built here turns that dataset into a searchable gallery and answers natural-language queries with three strategies whose differences are deliberate:

- **A** · `clip` — an OpenCLIP ViT-L-14 image index. No annotation is used. This is the zero-knowledge baseline: minutes of CPU and no human labour, but it sees the scene and not the joke.
- **B** · `clip_nlp` — CLIP fused with two annotation signals: BGE dense vectors over a four-field document, and field-weighted BM25, followed by a cross-encoder rerank. It is the upper bound when human annotation is available.
- **C** · `vlm` — Qwen2.5-VL reads each image offline and writes a structured description; BGE indexes those descriptions and fuses them with CLIP. It approximates annotation without a human, and generalises to unseen memes.

Because all three search the same images with the same queries, the same qrels and the same metric code, the resulting mAP differences are attributable to the representation rather than to three unrelated setups. That is the entire point of the design, and it shapes almost every decision described on the following pages.

> **Reading this document**
>
> Sections 3 and 7 correspond to pages 1–2 of the diagram set (architecture, then the three strategies side by side). Sections 8–10 correspond to pages 3–5 (build flow, query flow, evaluation). Each of those chapters closes with its diagram on a page of its own — the prose before it explains

> every box, so the two read as a pair. Section 13 lists the commands that regenerate everything, and Appendix B maps every figure back to the file it was read from.

2 The dataset and the gallery

Six thousand annotated memes become a 6,378-image gallery with a stable integer key, a bounded JPEG, and a manifest that everything downstream reads instead of the raw archive.

What Meme-Cap provides

For every meme the dataset supplies the source image, one or more literal image captions (`img_captions`), one or more intended-meaning captions (`meme_captions`), a title, the Reddit source URL and post id, and — for most memes — a list of metaphor/meaning pairs. Annotation length is modest — about 12.7 words of literal caption and 21.0 words of intended meaning per image — which is one reason a small text encoder (BGE-small, 384-d) is sufficient and a 512-token window never truncates an annotation document.

Preparation

`tsr prepare` is the only stage that downloads the *dataset*. Annotations come from the project's GitHub repository as two JSON files (trainval and test); images come from the HuggingFace mirror `hieupth/memecap` — a 1.7 GB zip — with a Google Drive fallback. The archive is streamed to disk with a `.part` suffix so an interrupted download resumes rather than restarting, and extraction is skipped when the recorded archive size still matches.

The other network dependency is model weights, fetched the first time an encoder is used and cached thereafter: OpenCLIP ViT-L-14 (~1.7 GB), BGE-small (~130 MB) and, when B's reranker is enabled, the cross-encoder (~1.1 GB). After that first load the whole system runs offline — which is also why the deployment image ships a model-cache volume rather than downloading per request.

Each source PNG is converted to RGB with transparent pixels composited onto white. This matters: a meme's white background is part of the image, and a transparent crop would render as black in some viewers and as white in others. The longest side is bounded at 768 px (never upscaled) and the result is written as JPEG quality 90. The bound is a compromise — it keeps overlaid text legible for the VLM's OCR field while producing a 492 MiB gallery instead of a 1.7 GB one.

De-duplication is by SHA-1 of the source bytes. Four byte-identical files are dropped, leaving **6,378 images from 6,382 annotations**. A perceptual dHash is computed and stored on every record but is deliberately *not* used to merge images: the gallery's identity is exact, and near-duplicate analysis can be added later without re-decoding anything.

The manifest

Every downstream stage reads `data/manifest.jsonl`, never the raw zip. One JSON record per image, written in `image_id` order:

Field	Meaning	Used by
`image_id`	the dataset's identifier, e.g. `memes_d079np`	everything; the stable key
`post_id`	Reddit post id	provenance links in the UI
`split`	`trainval` or `test`	the evaluation split guard
`path`	relative path of the normalised JPEG	image and thumbnail routes
`width`, `height`	dimensions after normalisation	layout and aspect handling
`sha1`	SHA-1 of the source PNG	de-duplication, VLM cache key
`phash`	perceptual dHash	future near-duplicate analysis
`title`	meme title, may be null	B's `title` field
`source_url`	original Reddit image URL	attribution in the detail panel
`img_captions`	literal scene captions	B's `scene` field
`meme_captions`	intended-meaning captions	B's `meaning` field
`metaphors`	metaphor / meaning pairs	B's `metaphor` field, metaphor queries
`category`	dataset category	reporting only

> **Why the split matters**
>
> The gallery is 5,820 trainval and 558 test images. Every evaluation query is drawn from the test split, and the freeze step re-checks that the source is still in it. Without that rule a strategy could be tuned on an image that a query later asks for, and the comparison would measure memorisation rather than retrieval.

`manifest.meta.json` sits beside the manifest with the record count, annotation count, duplicate count, the manifest SHA-256, and the normalisation parameters. The schema version is not there — it lives in each index's own `meta.json`, because the manifest can exist before any index does.

3 System architecture

Two bands that never run in the same process, joined by the artefacts they share: the offline band builds `data/`, the online band only reads it.

3.1 The offline band

Four steps produce the gallery. **Meme-Cap source** downloads annotations and the image archive. **Normalise** converts PNG to bounded JPEG. **De-duplicate** removes byte-identical files by SHA-1. **manifest.jsonl** records one normalised row per surviving image. Section 2 covers each step in detail; the important architectural point is that the manifest, not the archive, is the boundary — every later stage can be re-run without the 1.7 GB zip.

3.2 The artefacts

The band in the middle of the page is the interface between offline and online. Nothing crosses it except these files:

Artefact	Written by	Read by	Contents
`gallery.db`	`build --strategy gallery`	every strategy, every request	SQLite schema v2: `images`, `annotations`, `metaphors`, `vlm_docs`, `strategy_docs`, `nlp_docs`, `meta`, plus the FTS5 mirror
`index/clip/`	the same command	A, B and C	6,378 × 768-d image vectors, the id map, `meta.json`
`index/clip_nlp/`	`build --strategy clip_nlp`	B	384-d BGE vectors over the annotation documents
`index/vlm/`	`build --strategy vlm`	C	384-d BGE vectors over the VLM descriptions
`vlm/descriptions.jsonl`	`vlm-caption`	`vlm-caption` itself, as its resume cache	append-only log of captioning attempts

The panel is drawn between the two bands because it *is* the interface. The offline band only writes; the online band only reads; and because they are separate processes, captioning on a GPU host and serving on a laptop can run at the same time. The server needs the index files, not the caption cache, which is why a deployment only has to copy `data/index/vlm/`.

3.3 The online band

Browser. Seven page routes — `/`, `/clip`, `/clip-nlp`, `/vlm`, `/compare`, `/evaluation`, `/settings` — served from five HTML files in `web/`, because the three strategy routes share `search.html` and switch on a `data-strategy` attribute. They are plain HTML and vanilla JavaScript; all state comes from the JSON API, so changing the data never requires rebuilding a page.

FastAPI. Eighteen routes under `/api`: fourteen return JSON and four return files (two images, two HTML reports). `k` is limited to 1–100 and defaults to 12 on the search routes; `/api/compare` limits it to 1–50 and defaults to 10. Unknown strategy keys raise before any model is touched.

StrategyManager. Owns the SQLite store and constructs the three strategy objects at startup, loading no weights and no index files. Each strategy loads itself lazily through `ensure_loaded()`, recording either `ready` or a `load_error` string. Both `search()` and `stats()` trigger it, so the home page's `/api/health` and `/api/stats` calls validate every index before the first query, while encoder weights wait for the first search. A failure is reported through `/api/health`, never raised into the page. A process-wide lock serialises searches, because the encoders and FAISS indexes are shared mutable state.

The three strategies. All three read the *same* `index/clip/` vectors: A uses them alone, B fuses them with annotation signals, C fuses them with VLM-description signals. Holding the visual representation constant is what makes the comparison attributable.

SearchResponse. The payload every column returns — rank, fused score, per-signal components, captions, metaphors, the VLM summary, and the latency of that call. The components are what let the interface show *why* an image ranked where it did.

3.4 Four invariants

Two are drawn in the strip at the bottom of page 1; the other two are enforced by the build itself. Each prevents a specific silent failure.

#	Invariant	What would go wrong without it	Where
1	`images.id` is assigned once, in `image_id` order, and never renumbered	a strategy rebuild would shift ids and every stored qrel would point at the wrong image	page 1 strip
2	Every index has an id map; there is no `arange` fallback	a search would return FAISS row positions and silently answer with unrelated images	`index.py`
3	Every index writes `meta.json` and the loader validates it	vectors from a different model or manifest would be served as if they matched	page 1 strip
4	A strategy build compares a content hash of the stored annotations against the manifest	edited annotation text would keep the same ids, and the vectors would describe the old wording	page 3, Verify

The first is the load-bearing one. `images.id` is the only key shared by SQLite, the FAISS row ids and `image_ids.npy`. Strategy rebuilds rewrite `strategy_docs` and their own vector files; they never touch `images`. That is why an evaluation set frozen months ago still points at the right images after a rebuild.

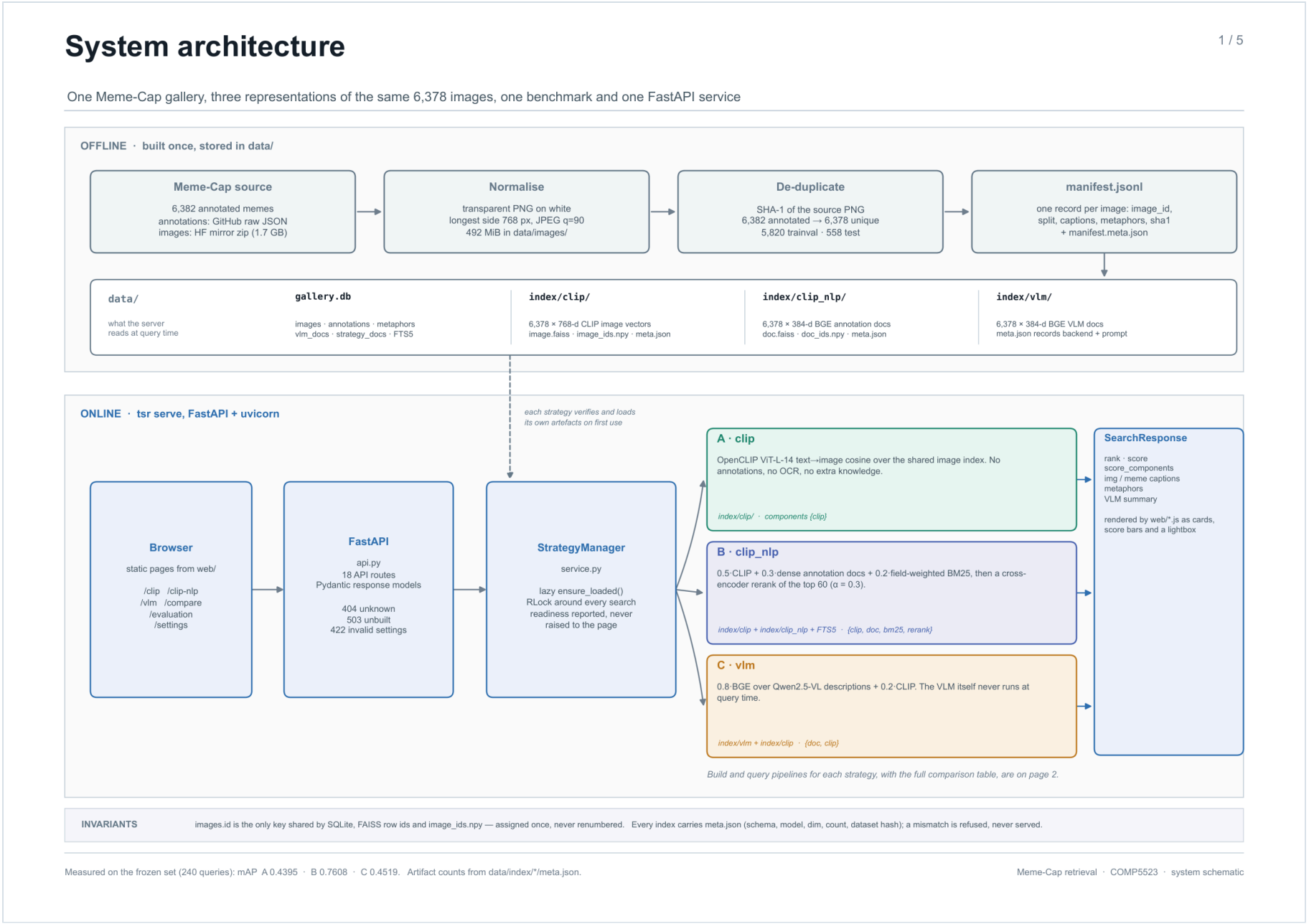


Figure 1. Page 1 of the diagram set. The offline band writes the gallery and the three indexes; the online band serves pages and JSON from them.

4 Strategy A — CLIP only

The zero-knowledge baseline: one image index, one text encoder, inner product. No annotation is read at query time and none feeds the embedding pass.

Build

The gallery is encoded once with the CLIP image tower (OpenCLIP ViT-L-14, `laion2b_s32b_b82k`) at batch 64. Every vector is L2-normalised before it is written, so an inner-product search is exactly cosine similarity. The index is stored as `index/clip/image.faiss` with `image_ids.npy` beside it and a `meta.json` recording the model, dimension, count, dataset hash and config hash.

> **A subtlety worth stating**
>
> The command that produces this index is also the command that inserts the image rows, and that insert reads the manifest's annotations into `gallery.db` so the other strategies can use them. Strategy A never reads those columns; it only shares the build. The claim that matters for the comparison is narrower and true: **no annotation text feeds A's embedding pass or its query path.**

Query

The query goes through the CLIP text tower (77 tokens) and is compared with all 6,378 image vectors by inner product. The strategy over-fetches `max(2k, k)` — a hard-coded 2× — and then drops every candidate whose similarity is ≤ 0, keeping the top *k*. The zero cut exists because CLIP text embeddings can be anti-correlated with an image; those candidates are confidently irrelevant and there is no reason to pad a result list with them.

Prompt ensembling (averaging several prompt templates) is available but off by default. The ablation in Section 11 measures it: it changes mAP by +0.002, which is well inside noise, so the default stays simple.

What it cannot see

The joke, and any text drawn on the image. A query describing the visible scene works. A query describing the meme's meaning works only when the meaning happens to be visible. This is not a defect to be fixed inside strategy A; it is the definition of the baseline, and every gain in B and C is measured against it.

Its cost profile is what makes it useful in production: minutes of CPU to build, a median of 14 ms per query, no annotation labour, and it generalises to any new image the moment the image is encoded.

5 Strategy B — CLIP + annotation NLP

Three candidate lists — pixels, dense annotation text, lexical annotation text — fused, then reranked by a cross-encoder that reads the query and the document together.

Build: one document, two indexes

Each image becomes four fields:

Field	Source	Construction
`scene`	`img_captions`	literal captions joined by `/`
`meaning`	`meme_captions`	intended-meaning captions joined by `/`
`metaphor`	`metaphors`	`"X means Y"` pairs joined by `;`
`title`	`title`	the dataset title, possibly empty

One correction happens before indexing. Meme-Cap marks a metaphor row with the meaning *"not related to the meme context"* when the "metaphor" is really a literal element of the picture. There are **1,968** such rows in the 6,378-image gallery, and indexing them would teach the ranker that *a man* is this meme's symbolism. They are dropped.

The four fields are written twice. As a single dense document —

```
scene: <literal captions>
meaning: <intended meaning>
metaphor: <X means Y; ...>
title: <title>
```

— embedded with BGE-small-en-v1.5 (CLS pooling, 384-d, 512 tokens) into `index/clip_nlp/`; and as four columns of an FTS5 table (`nlp_docs`, kept in sync by triggers) for BM25.

Query: three signals

All three signals over-fetch `max(5k, 100)` candidates, because a fusion can only reorder what it was given and 100 is a cheap floor at this gallery size.

Signal	Query representation	Index	Why it is there
clip	CLIP text tower	`index/clip/`	the visual channel; keeps scene queries working
doc	BGE with the query instruction	`index/clip_nlp/`	meaning that shares no words with the annotation
bm25	tokens, stop-words removed, lexicon-expanded	FTS5 over four columns	exact names, distinctive phrases, rare words

The BM25 column weights are `scene 1.0 · meaning 1.6 · metaphor 1.8 · title 0.6`. A retrieval query usually expresses what a meme *means*, so the meaning and metaphor columns count for more than a scene description; the title is short and noisy and counts for least.

QUERY EXPANSION STAYS LEXICAL

A hand-written lexicon maps meme phrases to descriptive terms — *"distracted boyfriend"* becomes couple, jealousy, distraction, ignoring. Those terms enter **only the BM25 query**, at weight 0.5, merged per image by taking the maximum. CLIP and BGE always see the original sentence. This is deliberate: an expansion is a guess, and a wrong guess must not be able to drag the semantic signals with it. A lexical expansion can only add candidates the literal query missed.

FUSION

Each signal is min-max normalised over its candidate union, so an image absent from a list scores 0 rather than being dropped. The combined score is

```
fused = 0.5 · clip + 0.3 · doc + 0.2 · bm25
```

with weights renormalised whenever a signal is disabled or returns nothing. That detail matters operationally: with CLIP unavailable B degrades to a two-signal ranker instead of returning zeros, and the ablation that removes CLIP entirely still reaches 0.780 mAP.

RERANKING

The top 60 candidates are scored by a cross-encoder (`BAAI/bge-reranker-base`) that sees the query and the document in the same forward pass, and the score is blended:

```
fused ← 0.7 · fused + 0.3 · rerank
```

This is the single most expensive and most valuable component. Removing it costs 0.072 mAP and drops median latency from 226 ms to about 43 ms — a five-fold saving that the default configuration declines to take.

What B depends on

Human annotation. It has no cold-start path: a new meme arrives with no captions and no metaphors, so B cannot rank it until someone annotates it. That limitation is not a bug to be patched — it is the cost that strategy C is built to approximate.

6 Strategy C — VLM descriptions

Qwen2.5-VL reads every image once, offline, and writes the annotation a human would have written. Serving never loads the 3B model — only a 384-d text encoder.

What the model is asked for

The prompt requires a single JSON object with six fields: `scene` (what is literally visible), `ocr_text` (all overlaid text, verbatim), `meaning` (the joke or message), `metaphors` (at most three pairs), `emotions` and `keywords` (five to ten). Two constraints are stated in the prompt: do not identify real people, and do not use outside knowledge of a meme template unless the picture supports it. Both exist to keep the description a description rather than a lookup of what the model remembers about the template.

Generation

Generation runs on a GPU host with bf16, 448 new tokens, and a pixel budget capped around 1024^2. `models.vlm.batch_size` controls images per call; the shipped default is 1, and the reported full run used 8 — roughly 1.5 s per image, about 2 h 35 min for the gallery.

Malformed JSON is repaired before it is rejected: the parser locates the outermost object span, fixes stray quotes and missing commas where it can, and re-parses. The first full pass still left **8 of 6,378 images (0.13%)** that local repair could not fix; re-running those images recovered all of them, and the final cache holds a `status="ok"` row for every image.

The cache and its key

Every attempt is appended to `vlm/descriptions.jsonl` and flushed to disk. The resume key is **(image_pk, image_sha1)**; `backend`, `model` and `prompt_version` are stored on every row and act as a validity filter. The distinction matters: an image whose bytes changed gets a new sha1 and a new key, while a prompt revision invalidates old rows without changing the key. Because the file is append-only, a retried image can carry an earlier `error` line as well as its later `ok` line — the loader keeps only the last successful entry.

Build

The six fields are flattened into one document embedded with the same BGE model as B:

```
scene: ... / overlaid text: ... / meaning: ... / metaphors: ... / emotions: ... / keywords: ...
```

Only rows with `status="ok"` and the *current* provenance triple are read. That triple is recorded in the index's `meta.json` and checked again at load time, so descriptions from two different generators can never be mixed without anyone noticing.

Two gates specific to C

> **Coverage and provenance**
>
> The index must cover at least **95%** of the gallery. An interrupted captioning run would otherwise present itself as a built strategy and quietly rank only the images it processed. When coverage falls short, C reports as unavailable with the exact counts rather than serving a partial gallery. The second gate is the provenance triple above; loading refuses a mismatch.

Query

The query is encoded with BGE and searched against `index/vlm/`, over-fetching `max(3k, 100)` candidates, then fused with CLIP:

```
fused = 0.8 · doc + 0.2 · clip
```

When CLIP is unavailable the document weight becomes 1.0 rather than leaving a fifth of the score unaccounted for. Reranking is off by default for C. The evidence that a cross-encoder helps comes from B's annotation documents (Section 11.3); on C it would add roughly the

same 180 ms it adds to B — 226 ms with it, 43 ms without — to a strategy that otherwise answers in 35 ms, and no ablation covers that trade for C. The pure-VLM variant is therefore the cleaner measurement of the description channel on its own.

What C costs and what it buys

A one-off GPU cost of roughly 2.5 hours and no human annotation. The artefact that reaches serving is small: the VLM document index is 9.4 MiB for 6,378 × 384-d vectors, against 18.7 MiB for the CLIP image index. It generalises to unseen memes, which B cannot do. What it does not buy is B's accuracy: a 3B model compresses an image to its gist, so fine visual detail is lost — the next section quantifies exactly that trade.

7 The three strategies side by side

Same gallery, same CLIP image vectors, same API. The only difference is which text, if any, represents an image at query time.

7.1 What actually differs

	A · clip	B · clip_nlp	C · vlm
Text per image	none	human captions and metaphors	model-written description
Signals at query time	1 — CLIP	3 + rerank	2 — doc + CLIP
Candidates fetched	`max(2k, k)`	`max(5k, 100)` per signal	`max(3k, 100)`
Post-processing	drop score ≤ 0	cross-encoder on top 60, α = 0.3	none by default
One-off cost	minutes of CPU	human annotation of 6.4k images	~2 h 35 min of GPU
Works on unseen memes	yes	**no** — they arrive unannotated	yes
Median latency	14 ms	226 ms	35 ms
mAP on the frozen 240	0.4395	0.7608	0.4519

7.2 The two ablations that make the difference concrete

B is slightly better without CLIP (0.7608 → 0.7802). Once human annotations are indexed, a 0.5 weight on pixels dilutes a stronger text signal. The weight was fixed before evaluation and never tuned on the test set; this is the clearest tuning opportunity in the system, and it is reported as an ablation rather than quietly applied.

C is much worse without CLIP (0.4519 → 0.3922). A VLM description compresses the image to its gist and drops visible detail, so the pixels have to supply what the text omitted. The same channel that is noise for B is signal for C.

> **Why that asymmetry is the interesting result**
>
> If VLM text were simply a cheaper copy of annotation text, both ablations would point the same way. They do not. Human annotation and model description cover different parts of the query space: annotation carries the meaning a person chose to write down, while a description carries what a model can see and name. Section 11 shows the same split in the per-category table — C gains on intent and metaphor but loses on visual detail, which is why it lands beside A rather than beside B.

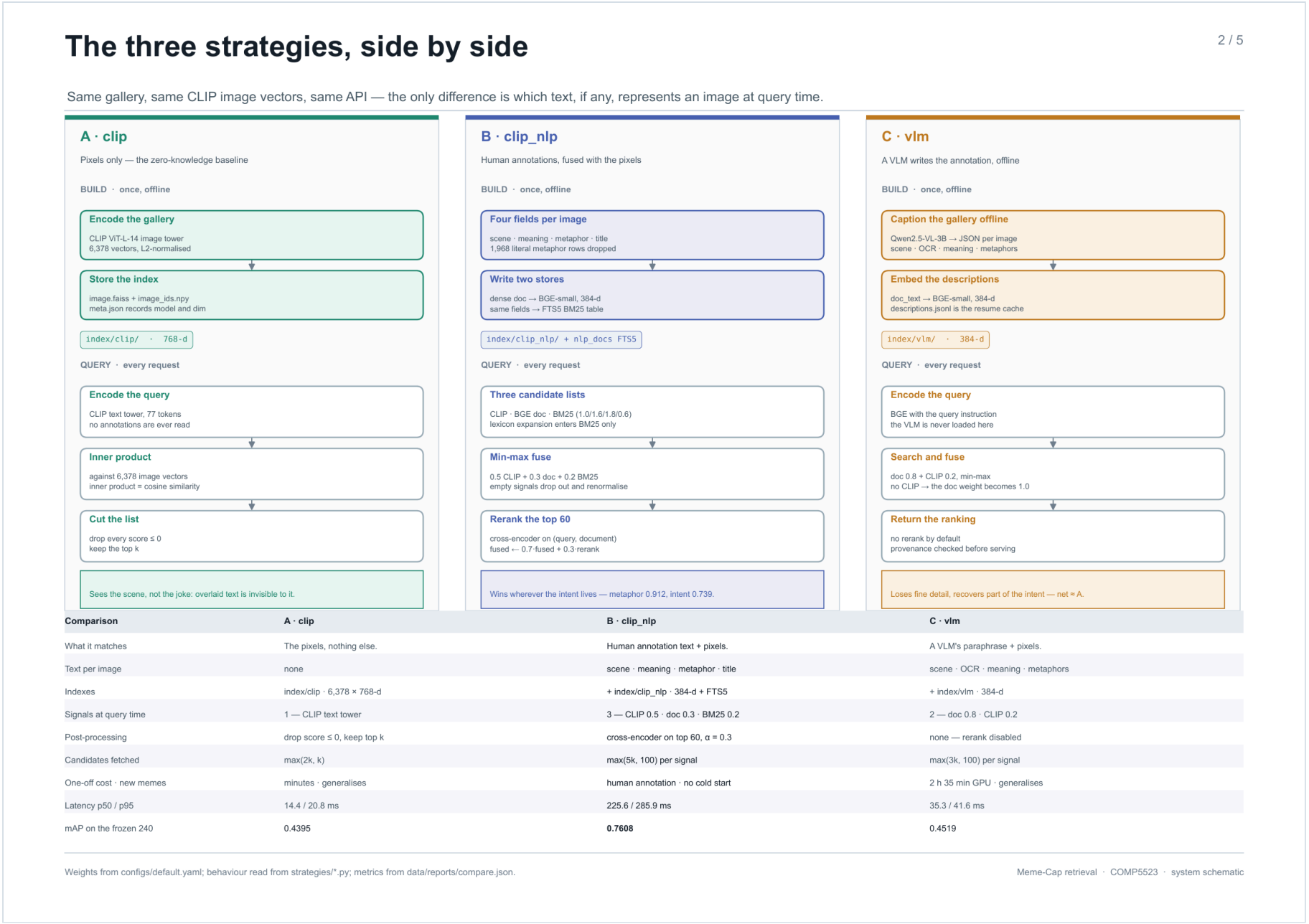


Figure 2. Page 2 of the diagram set. Build and query pipelines for each strategy, with the comparison table.

8 Offline build flow

Five commands turn the Meme-Cap download into the gallery and the three indexes. Every stage is keyed by a content hash, so a stale artefact is refused rather than served.

8.1 `tsr prepare` — once, network required

Downloads the annotation JSON and the image archive (resumable), extracts `memecap/images/*.png` and `memecap/caption/*.json`, normalises, de-duplicates by SHA-1 and writes the manifest. *Guards:* an existing manifest is reused unless `--force`; extraction is skipped when the recorded archive size still matches, so re-running after a failure does not re-extract 6,416 files.

8.2 `tsr build --strategy gallery` — once

Inserts the 6,378 image rows in `image_id` order inside one transaction, then encodes every image with CLIP. *Guards:* the build asserts that ids are contiguous, that the vector count matches the row count, and that a content hash of the stored annotations matches the manifest. Any failure aborts **before** an index is written. The alternative — writing an index whose row i is a different image than id i — is the one error no downstream check would catch, because every later stage trusts the id map.

8.3 `tsr build --strategy clip_nlp`

Builds the four annotation fields, writes `nlp_docs` with its FTS5 triggers, and encodes the dense documents into `index/clip_nlp/`. *Guard:* the 1,968 literal-metaphor rows are excluded; without that rule, *a man* would be indexed as this meme's symbolism.

8.4 `tsr vlm-caption` — GPU host, hours (~2 h 35 min)

Batched generation, JSON repair, upsert into `vlm_docs` and append to the JSONL cache with a flush per line. A failed batch is retried image by image, so one unreadable file cannot stall the run. *Guard:* the resume key is image + SHA-1, and the provenance triple on every row rejects rows from a different generator.

8.5 `tsr build --strategy vlm`

Filters `vlm_docs` to `status="ok"` and the current provenance triple, encodes `doc_text` with BGE, and records the triple in `meta.json`. *Guard:* a different generator, or coverage under 95%, makes C read as unavailable.

8.6 Ordering, and what a rebuild may touch

`--strategy all` runs clip → clip_nlp → vlm. It skips C with a warning when `models.vlm.backend` is `none`, and also when its build raises a `RuntimeError` — no `vlm_docs` match the configured triple, or the gallery no longer matches the manifest. B and C never touch `images`: they rewrite only their own rows and vector files, which is what keeps `images.id` stable across rebuilds.

`python -m tsr stats --json` prints readiness, row and vector counts and the model string for every stage — the quickest way to see which index is built and what provenance it records.

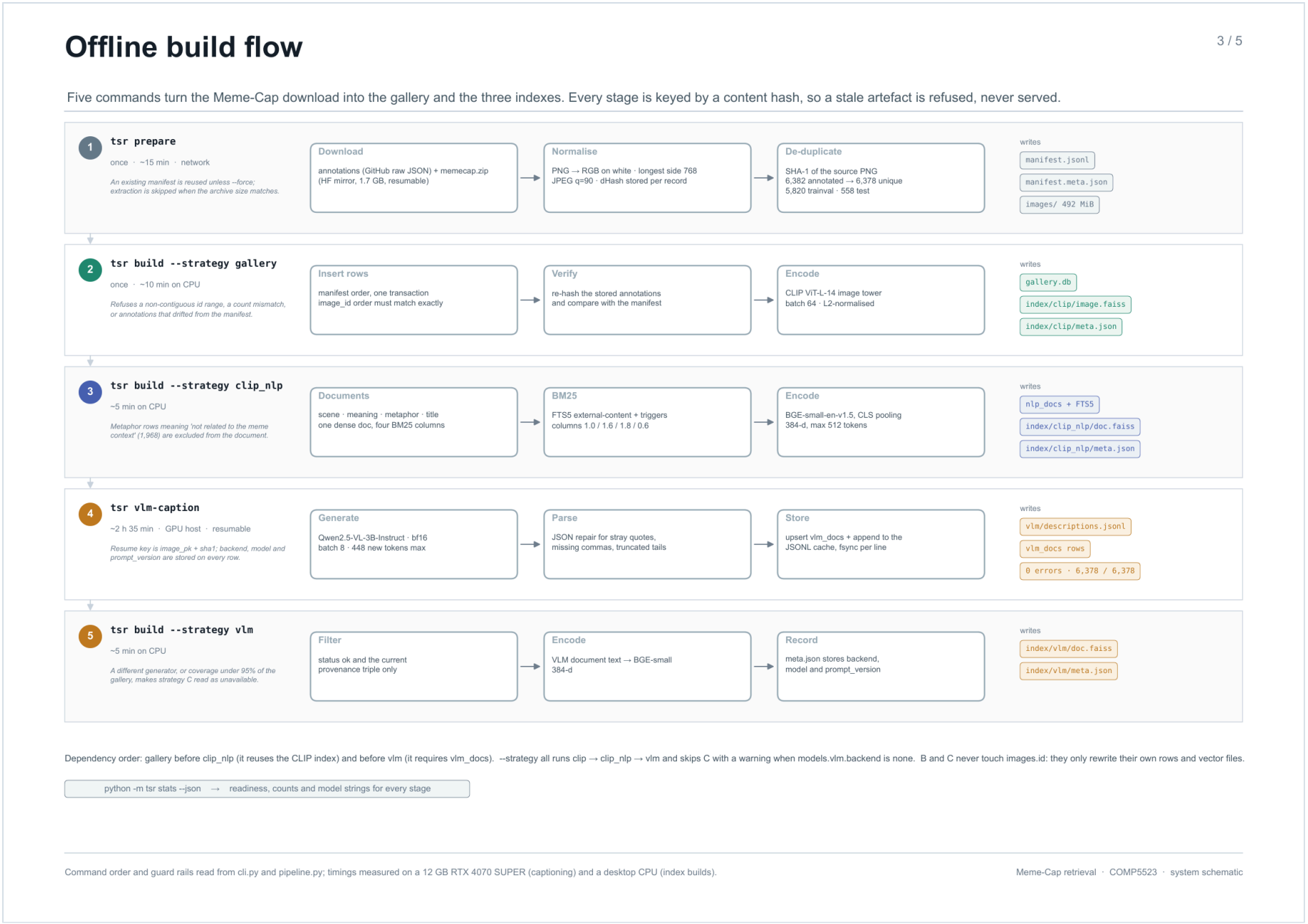


Figure 3. Page 3 of the diagram set. The five build stages, what each writes, and the guard on each.

9 Online query flow

One request, from the page to the ranked payload. Models and indexes load on the first query of each strategy, never at process start.

9.1 The nine steps

1. The page calls `GET /api/strategies/{key}/search?q=…&k=…`.
2. The route calls `StrategyManager.search(key, query, k)`.
3. The manager resolves the strategy — an unknown key is a 404 — and calls `ensure_loaded()`.
4. The strategy encodes the query: CLIP's text tower for A, BGE with the query instruction for B and C.
5. It searches its indexes, fetching `max(2k, k)` for A and `max(mult·k, 100)` for B (5×) and C (3×).
6. The index returns ids and scores; inner product over L2-normalised vectors is cosine similarity.
7. The strategy returns hits with per-signal components and its own timing.
8. The manager enriches the hits from `gallery.db` — captions, metaphors, title, source URL, VLM summary — because an index only knows ids and scores.
9. The route returns a `SearchResponse`; the page renders cards, score bars and a lightbox.

9.2 Failure semantics

Status	When	What the caller sees
404	unknown strategy key — the set is closed to `clip`, `clip_nlp`, `vlm`	a JSON detail naming the key that was rejected
503	known but not ready: index missing, stale, or failing a coverage or provenance gate	the strategy's `load_error` string
422	invalid payload — empty query, out-of-range setting	a validation detail

`/api/compare` never fails as a whole. A column that cannot answer is returned as `{"ready": false, "error": …}` and the other columns still serve. That is what lets the comparison page stay useful while strategy C is still being captioned — a design decision made because the alternative, failing the whole request, hides the two working strategies.

9.3 What loads when

Startup loads configuration, opens SQLite and constructs the three strategy objects — no model weights, no FAISS files. The first query of a strategy loads its encoder, its indexes and, for B, the cross-encoder reranker. The load is guarded by `ensure_loaded()`, so the cost is paid once and never repeated. The VLM stack is never loaded by the serving process at all: captioning is an offline job, and the serving image deliberately does not ship those extras.

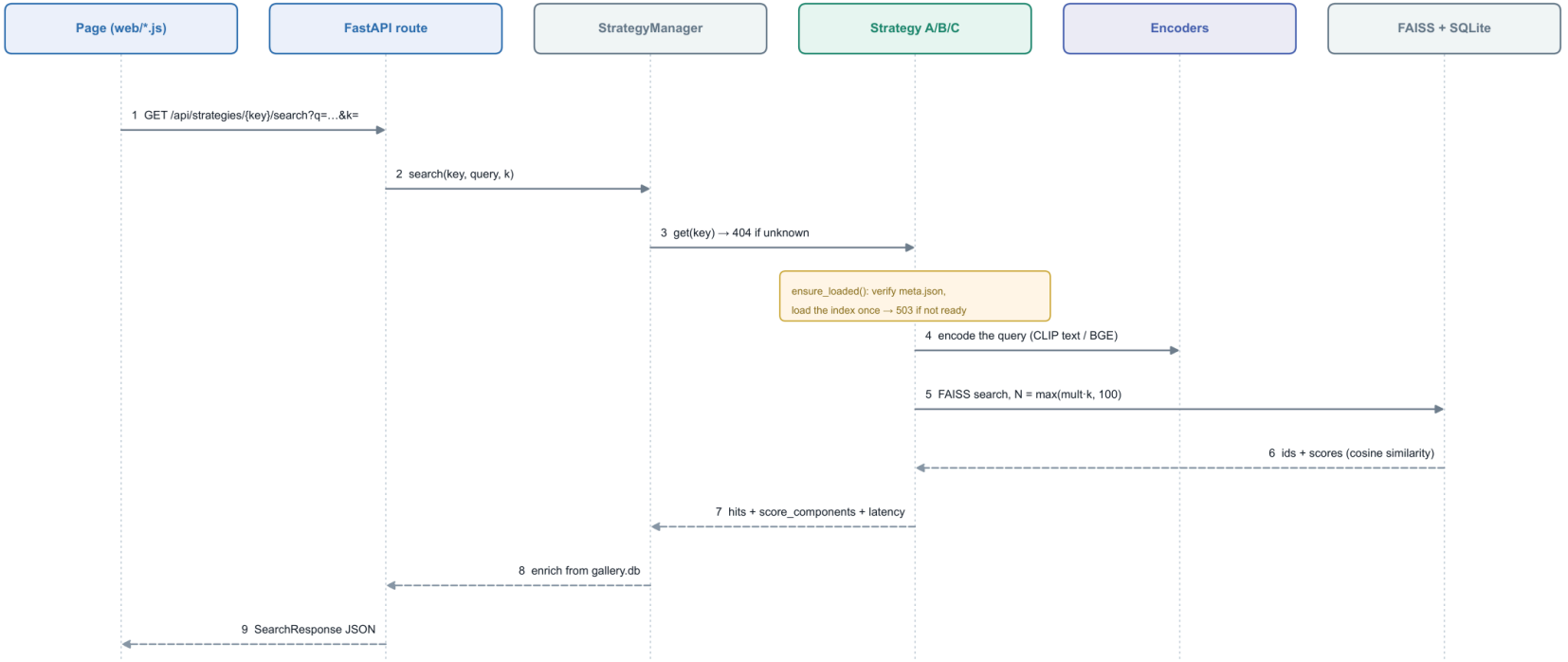
9.4 The latency budget

Strategy	p50	p95	Where the time goes
A · clip	14 ms	21 ms	one text encode, one flat inner-product scan
B · clip_nlp	226 ms	286 ms	three index lookups plus a cross-encoder over 60 pairs
C · vlm	35 ms	42 ms	one text encode, two index lookups, no rerank

B's extra time is almost entirely the cross-encoder. Turning it off drops latency to roughly 43 ms at a cost of 0.072 mAP; all three configurations sit far inside an interactive budget, which is why the default keeps the reranker on. The lesson is not that reranking is cheap — it is that 226 ms is cheap enough here, and the ablation makes the trade explicit rather than hiding it.

Online query flow

One search request, from the page to the ranked payload. Models and indexes load on the first query of each strategy, never at process start.



Route order from api.py, readiness checks from service.py and strategies/vlm.py; latencies from data/reports/compare.json.

Meme-Cap retrieval · COMP5523 · system schematic

Figure 4. Page 4 of the diagram set. The nine steps of a search request, failure semantics, lazy loading and measured latency.

10 Evaluation

A frozen, leakage-checked benchmark run through every ready strategy, with the same gallery and the same metric code — and a report that the CLI and the web page both read.

10.1 Building the benchmark

240 queries are drafted for 80 visual, 80 intent and 80 metaphor sources, all drawn from the 558-image test split. Each draft is rewritten by hand and must clear the leakage gate before it is frozen. The gate rejects a query when:

- an 8-gram overlaps **any** annotation field of the source **or of any gallery image**;
- the query embeds a whole short annotation (≥ 4 tokens) — no 8-gram can see a three-word caption pasted verbatim;
- a contiguous run of ≥ 5 tokens is copied from the source caption — a short copy inside a longer sentence slips under the n-gram rule;
- token Jaccard with the source caption is ≥ 0.6 — this rule has no length floor, so short captions cannot be pasted either.

The freeze also verifies that every source is still in the test split and that its annotation snapshot is unchanged, and it aborts rather than writing a partial set if any reviewed query is rejected. Every verdict is stored in `metadata.leakage`, so the set can be audited without re-running the gate.

> **Why B makes leakage the central problem**
>
> B indexes human annotations. If a query were allowed to reuse an annotation's wording, B would be scored on recognising text it has already stored — and the metric would measure string overlap, not retrieval. The gate is not a formality; it is what makes the comparison meaningful, and Section 11 shows exactly how much a leaky set would have flattered B.

10.2 Measuring

All three strategies run the same queries against the same gallery with `depth = max(top_k, largest relevant set, 100)`, so recall and mAP are never truncated by the reported @k cuts. Every strategy is scored on P@1/5/10, R@1/5/10, mAP, MRR, nDCG@10, median rank and misses; the results table reports the headline subset.

Relevance is graded. Grade 2 is the source meme. Grade 1 is assigned only by deterministic rules: a near-identical image caption (token Jaccard ≥ 0.8), an identical intended meaning, or a shared non-literal metaphor that carries the same meaning and appears in at most 10 images. No model judges relevance — "looks similar to a model" must never become ground truth.

10.3 Comparing

A paired bootstrap (1,000 resamples, fixed seed 42) over per-query metric values gives the confidence intervals. Holm–Bonferroni is applied once across the three pairwise × five metric tests, so the three comparisons form a single family, and p-values are floored at the bootstrap resolution 0.001.

`--ablations` re-runs the benchmark with seven single changes, each isolating one design decision: prompt ensembling for A; B without BM25, without the dense channel, without CLIP or without the reranker; and C as pure VLM or with the CLIP weight raised to 0.35.

The run writes `data/reports/compare.json`, renders it to `compare.html` and `failures.html`, and serves the same JSON through `/api/eval/summary`, which the `/evaluation` page reads. The CLI and the page therefore cannot disagree.

Evaluation and reporting flow

5 / 5

A frozen, leakage-checked benchmark is run through every ready strategy; the same report files drive the CLI and the /evaluation page.



LEAKAGE DIAGNOSTIC The untouched human annotations are also run as queries and reported separately (B +0.139 mAP, A −0.085, C −0.193). Leaky by construction — never mixed into the main table.

MEASURED RESULT · 240 frozen queries, identical gallery and retrieval depth

Strategy	mAP [95% CI]	MRR	nDCG@10	P@1	R@10	p50
A · clip	0.4395 [0.385, 0.498]	0.4461	0.4744	0.358	0.607	14 ms
B · clip_nlp	**0.7608 [0.713, 0.808]**	0.7647	0.7804	0.700	0.870	226 ms
C · vlm	0.4519 [0.399, 0.507]	0.4632	0.4881	0.383	0.626	35 ms

B − A mAP +0.321 [0.270, 0.371] p_holm = 0.015
B − C mAP +0.309 [0.255, 0.363] p_holm = 0.015
C − A mAP +0.012 [−0.038, 0.064] not significant

Per-category mAP (visual / intent / metaphor) · A 0.315 / 0.444 / 0.559 B 0.631 / 0.739 / 0.912 C 0.247 / 0.539 / 0.570

Ablations (mAP) · B − BM25 0.704 · B − dense 0.704 · B − CLIP 0.780 · B − reranker 0.689 · C pure VLM 0.392 · C + CLIP 0.35 0.472

Reading B wins by +0.32 mAP over both annotation-free strategies; C and A are statistically indistinguishable, so a 3B VLM is a cheap approximation of the annotation signal, not a substitute.

Sources: data/eval/memecap_eval_v1.{json,qrels.jsonl} and data/reports/compare.json (1,000 bootstrap resamples, seed 42).

Meme-Cap retrieval · COMP5523 · system schematic

Figure 5. Page 5 of the diagram set. Benchmark construction, measurement, comparison, and the measured result.

11 Results

240 frozen queries, one gallery, identical retrieval depth. B is far ahead; C is statistically indistinguishable from A, and the per-category split explains why.

11.1 Headline

	mAP	MRR	nDCG@10	P@1	R@10	median rank	misses	p50
A · clip	0.4395	0.4461	0.4744	0.358	0.607	2	46	14 ms
B · clip_nlp	**0.7608**	0.7647	0.7804	0.700	0.870	1	13	226 ms
C · vlm	0.4519	0.4632	0.4881	0.383	0.626	2	38	35 ms

B − A is +0.321 [0.270, 0.371] · p_{holm} = 0.015 mAP and **B − C** is +0.309 [0.255, 0.363] · p_{holm} = 0.015; both are significant on every metric after correction. **C − A** is +0.012 [−0.038, 0.064] · p_{holm} = 1.000 — not significant on any metric. The two annotation-free strategies are indistinguishable, which is itself the main result about strategy C: a 3B model recovers part of the meaning channel but nets out to roughly the pixel baseline.

11.2 Per category

	visual	intent	metaphor
A · clip	0.315	0.444	0.559
B · clip_nlp	0.631	0.739	0.912
C · vlm	0.247	0.539	0.570

B improves every category — visual 0.631 vs 0.315 for A and 0.247 for C, intent 0.739, metaphor 0.912 — with its largest gain over A on metaphor and its smallest on intent, because its documents contain the human-written meaning and the metaphor pairs. C gains on intent and metaphor over A but **loses on the visual category**: a 3B model describes the gist and the overlaid text, not every visible detail.

11.3 Ablations

Variant	mAP	p50
clip + prompt ensemble	0.4415	7 ms
clip_nlp - BM25	0.7040	198 ms
clip_nlp - dense doc	0.7043	200 ms
clip_nlp - CLIP	0.7802	228 ms
clip_nlp - reranker	0.6887	43 ms
vlm only (no CLIP)	0.3922	8 ms
vlm + CLIP 0.35	0.4720	18 ms

Three results carry the argument. Removing B's reranker costs the most of any single channel (0.7608 → 0.689), which is why it stays on despite dominating latency. Removing CLIP from B *raises* mAP, so the visual channel is dilution once annotation text is present. Removing CLIP from C is the opposite. Together they show the two text sources are not interchangeable.

11.4 Worked examples

Three frozen queries, one per category, chosen where both annotation-free strategies put the source outside their top ten and B ranks it first. They are representative rather than cherry-picked: the pattern repeats across the 138 frozen queries on which B ranks the source higher than A.

VISUAL QUERY · Q0005

"a man biting his lip and lifting his eyebrows with a pained face after something awkward happened"

Strategy	Rank of the source	AP	RR
A · clip	—	0.000	0.000
B · clip_nlp	1	1.000	1.000
C · vlm	—	0.000	0.000

INTENT QUERY · Q0090

"the feeling of being knocked down by a mocking reply after you shared something sincere"

Strategy	Rank of the source	AP	RR
A · clip	—	0.000	0.000
B · clip_nlp	1	1.000	1.000
C · vlm	—	0.000	0.000

METAPHOR QUERY · Q0209

"an angry puffed up face standing in for a movie sequel built around the opposite of a smile"

Strategy	Rank of the source	AP	RR
A · clip	—	0.000	0.000
B · clip_nlp	1	1.000	1.000
C · vlm	77	0.013	0.013

In the visual case, the query describes a face and an expression; CLIP matches other faces and expressions, while B's annotation document contains the literal caption of the right image. In the intent and metaphor cases, the words of the query are not in the image at all — only in the annotation — so no pixel-only representation can reach them.

11.5 The leakage diagnostic

Strategy	Main set mAP	Raw-caption set mAP	Δ
A · clip	0.4395	0.3546	−0.085
B · clip_nlp	0.7608	0.8998	+0.139
C · vlm	0.4519	0.2591	−0.193

Running the untouched human annotations as queries is leaky by construction — B indexes those annotations — so it is reported separately and never enters the main table. Only B improves when queries copy annotation wording, and the gain is large enough to reorder the

strategies on its own. That is precisely why the frozen set uses rewritten queries, and why the comparison refuses to publish a set in which any query failed the gate.

11.6 Failure analysis

`failures.html` shows the ten worst queries per strategy with the graded relevant set and the top-5 retrieved images. A's and C's ten worst are all visual — fine-grained structure that a single embedding cannot carry, such as two-panel layouts, a specific expression or a costume. B's split evenly, five visual and five intent queries about abstract meaning, and its 13 misses overall spread across the categories (7 intent, 5 visual, 1 metaphor). Intent, not visual, is B's largest remaining gap.

12 Operating and configuration

Structural choices need a rebuild and live in configuration; ranking knobs are editable from the settings page and take effect on the next search.

12.1 Where a setting lives

Kind	Lives in	Examples	Change requires
Build-time	`configs/default.yaml`	model names, index type, dataset bounds, VLM backend and prompt version	a rebuild of the affected index
Runtime	`data/settings.json`, edited at `/settings`	fusion weights, BM25 weight, fetch multipliers, expansion, rerank top-N and α, C's CLIP weight, prompt ensembling, default *k*	nothing — applied on the next search

Runtime edits are re-applied at startup from `data/settings.json`. One exception matters: `app.default_top_k` is read once when the app is created and bound as the search route's query default, so changing it through the settings endpoint has no effect until the process restarts. Every other knob is read per request.

12.2 Rebuild rules

Rebuild an index when the dataset, the model or the document text changes; then restart the API. Readiness is a load-time snapshot, so a running process keeps serving the indexes it loaded. Stats are not a snapshot: strategies A and B re-read `meta.json` and the live row count on every request, so after a rebuild they describe the newer files on disk even though search still uses the old loaded index. Only strategy C reports the artefacts it actually loaded.

> **The practical consequence**
>
> After rebuilding, check `/api/health` before trusting a search result. If the process has not been restarted, it is still serving the previous index — and the stats page will look newer than the answers. This is the one place where the system's lazily-cached design can mislead, and it is why the readiness flag is reported per strategy rather than inferred from the files.

12.3 Deployment shape

The serving container needs the CLIP and BGE encoders, the index files and `gallery.db`. It does not need the VLM weights — captioning is offline, and the base image does not install those extras.

Enabling strategy C on a host therefore means shipping two things, not one: the vector files in `data/index/vlm/` *and* the matching rows in `gallery.db`. The build step reads `vlm_docs` from the database — it does not read `descriptions.jsonl` — and `vlm-caption` is the only command that ingests the JSONL, which it does with the VLM loaded. So on a GPU-less serving host, copy the index and the `vlm_docs` rows together (or copy `gallery.db` whole); do not expect "caption cache plus one build" to work there.

13 Reproducing the results

Every number in this document is produced by the commands below; the full run takes a few hours on a single GPU host, most of it captioning.

```
# small end-to-end loop (network required, ~300 images)
python -m tsr prepare --limit 300
python -m tsr build --strategy gallery
python -m tsr build --strategy clip_nlp
python -m tsr make-eval --num 60
python -m tsr eval --strategy clip,clip_nlp
python -m tsr serve                    # http://localhost:8000

# full dataset (6,378 images)
python -m tsr prepare
python -m tsr build --strategy gallery
python -m tsr vlm-caption              # GPU host, resumable, ~2 h 35 min
python -m tsr build --strategy all
python -m tsr eval --strategy all
python -m tsr compare --ablations

# checks
PYTHONPATH=src pytest
PYTHONPATH=src ruff check src tests
```

13.1 Determinism

The dataset is fixed by the manifest hash recorded in every index's `meta.json`. The query set is frozen in `data/eval/` and carries its leakage verdicts. The bootstrap uses a fixed seed. The only non-determinism in the reported pipeline is GPU generation for strategy C, which is why the descriptions are cached and the build consumes the cache rather than the model: re-running the build cannot change a single vector.

13.2 What to check after a rebuild

Check	Command	Expected
Gallery size	`tsr stats --json`	6,378 images, CLIP index ntotal = 6,378
Strategy readiness	`GET /api/health`	every built strategy `ready`, C only above 95% coverage
Provenance	`data/index/*/meta.json`	matching backend/model/prompt triple for C
Report consistency	`/evaluation` vs `data/reports/compare.json`	identical numbers

A File map

Where each responsibility lives.

```
configs/default.yaml       all build-time defaults
src/tsr/
  __main__.py              makes `python -m tsr` work
  cli.py                   argparse entry point, sub-commands
  config.py                AttrDict config + TSR_* environment overrides
  logging_utils.py         log formatting and verbosity
  memecap.py               download, extract, normalise, manifest
  manifest.py              record type, JSONL I/O, hashing
  store.py                 SQLite schema v2 + FTS5 BM25
  index.py                 FAISS index with a mandatory id map and meta checks
  artifacts.py             meta.json read/write and validation
  pipeline.py              gallery and strategy build pipelines
  encoders/clip.py         OpenCLIP image and text towers
  encoders/text.py         BGE encoder, CLS pooling, query instruction
  encoders/vlm.py          Qwen2.5-VL captioning, JSON repair, resume cache
  nlp/analyze.py           tokenisation, stop-words, lexicon expansion
  nlp/fusion.py            min-max, z-score, RRF
  nlp/rerank.py            cross-encoder
  strategies/base.py       Hit, SearchOutput, StrategyContext, Strategy
  strategies/clip_only.py  A
  strategies/clip_nlp.py   B
  strategies/vlm.py        C
  service.py               lazy StrategyManager
  evaluate.py              metrics, leakage checks, bootstrap
  compare.py               comparison, ablations, HTML reports
  api.py                   FastAPI routes
  schemas.py               response models
  settings.py              runtime-editable knobs
web/                       5 HTML pages + shared JS/CSS (7 page routes)
tests/                     unit tests, heavy deps importorskip-guarded
docs/                      this document, the diagram set, the report
```

B Where every figure comes from

Every table, figure and worked example is generated from the file named beside it at build time; the prose repeats those values and was checked against the same sources.

Figure	Source
6,382 annotations → 6,378 images, 4 duplicates	`data/manifest.meta.json`
5,820 / 558 split	`data/manifest.jsonl`, `gallery.db`
768 px, quality 90 (492 MiB once measured)	`configs/default.yaml` · `dataset.*`; size from `data/images/`
Model names and dimensions	`data/index/*/meta.json`
Fusion weights, BM25 column weights, fetch multipliers, rerank settings	`configs/default.yaml` · `strategies.*`
Build guards — id contiguity, content hash, provenance, 95% coverage	`src/tsr/pipeline.py`, `src/tsr/strategies/*.py`
Latencies, mAP, CIs, Holm-adjusted p-values, ablations	`data/reports/compare.json`
Query set, qrels and leakage verdicts	`data/eval/memecap_eval_v1.json`
Worked-example ranks and AP	`data/reports/compare.json` · `per_query`
Captioning throughput and parse failures	`data/vlm/descriptions.jsonl`, `vlm_docs`

Regenerate this document with `python docs/make_detailed.py`; it re-renders the five diagram pages from `docs/system-schematic.pdf` and re-reads every table above.

C Glossary

Terms used with a specific meaning in this system.

Term	Meaning here
gallery	the 6,378 normalised images plus their rows in SQLite; the single set every strategy searches
strategy	one of the closed set `clip`, `clip_nlp`, `vlm`; unknown keys are 404, known-but-unbuilt are 503
manifest	`data/manifest.jsonl` — the normalised record per image; the source of truth downstream
id map	`image_ids.npy` beside the CLIP index, `doc_ids.npy` beside the B and C document indexes; row *i* of the index belongs to this image id
meta.json	per-index sidecar: schema version, kind, model, dimension, count, dataset hash, config hash, build time
content hash	a hash of the stored annotations, compared with the manifest so edited text cannot keep old vectors
provenance triple	backend / model / prompt_version for VLM rows; a mismatch makes strategy C unavailable
coverage	the fraction of gallery images a strategy has a document for; C requires ≥ 95%
fused score	the combined, normalised ranking score; the per-signal parts are exposed as `score_components`
fetch multiplier	how many candidates a signal retrieves before fusion — 2× for A, 5× for B, 3× for C, with a 100 floor for B and C
leakage gate	the set of rules that rejects an evaluation query copying annotation wording
qrel grade	2 for the source meme, 1 for a deterministic near-equivalent, 0 otherwise
frozen set	`memecap_eval_v1` — 240 reviewed queries plus graded qrels, never edited after freezing
p_{holm}	a p-value after Holm–Bonferroni correction across the three pairwise × five metric tests